Anatomy of a Session Type Checker: The BICA Reborn Annotation Processing Pipeline

Alexandre Zua Caldeira
LASIGE, Faculdade de Ciências, Universidade de Lisboa
Lisbon, Portugal
azcaldeira@fc.ul.pt

Abstract

We describe the seven-stage annotation-processing pipeline of Bica Reborn, a Java session type checker for objects with parallel composition. A single `@Session` annotation triggers a chain of analyses—parsing, termination, well-formedness, state-space construction, conformance, lattice verification, thread safety, and typestate checking—each of which can reject a program at compile time. We catalogue every class of error that can arise at each stage, provide concrete examples that trigger each violation, and explain the rationale behind the ordering. The goal is to give practitioners an actionable reference for writing well-typed session-annotated Java classes.

1 Introduction

Session types specify the valid sequences of operations on an object. Bica Reborn brings session types to Java via a single annotation:

```
@Session("open . &{read: close . end, write: close .
    end}")
public class FileHandle { ... }
```

The string inside `@Session` is a *session type*—a grammar-like specification of allowed method-call orderings. At compile time, a Java annotation processor parses this string and runs seven analyses. If all pass, the program is accepted; if any fails, the compiler emits an error pointing to the annotation.

This article walks through every stage of the pipeline, explains what each check verifies, and provides concrete examples of inputs that trigger every class of error. The pipeline processes over 1,050 unit tests and validates 34 benchmark protocols drawn from real-world systems (SMTP, HTTP, gRPC, ATM, two-buyer, Kafka, MQTT, etc.).

2 Pipeline Overview

Figure 1 shows the pipeline. Each stage either succeeds (passing data to the next) or fails (emitting a compiler error and halting). A successful run through Stages 1–6 produces a verified state space and a classification of concurrent methods. Stage 7 then scans *client code* for protocol violations.

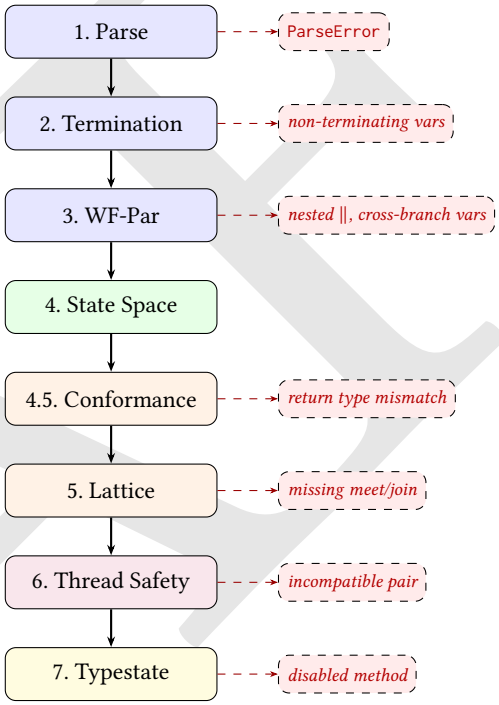

Figure 1. The Bica Reborn annotation-processing pipeline. Solid arrows show data flow; dashed arrows show error exits. Stages 1–3 operate on the AST, Stage 4 constructs the state space, and Stages 4.5–7 analyze it.

3 Stage 1: Parsing

The parser consumes the session type string and produces an immutable AST built from seven node types (`End`, `Var`, `Branch`, `Select`, `Parallel`, `Rec`, `Sequence`). The grammar is:

$$
\begin{array}{rcll}
S & ::= & \&\{m_1 : S_1, \ldots, m_n : S_n\} & \text{(branch)} \\
 & | & \oplus\{l_1 : S_1, \ldots, l_n : S_n\} & \text{(selection)} \\
 & | & (S_1 \parallel S_2) & \text{(parallel)} \\
 & | & \mu X.\, S & \text{(recursion)} \\
 & | & X & \text{(variable)} \\
 & | & \textbf{end} & \text{(terminated)} \\
 & | & S_1 \,.\, S_2 & \text{(sequencing)}
\end{array}
$$

Sequencing desugars: $m.S$ becomes $\&\{m : S\}$ (a single-choice branch).

3.1 Parse Error Catalogue

Table 1 lists every category of parse error with an example input that triggers it.

Table 1. All parse error categories, with triggering input and error message.

Category	Input	Error message
Empty input	`""`	unexpected token EOF
Trailing content	`"end end"`	unexpected token IDENT after expression
Unclosed brace	`"&{m: end"`	expected RBRACE, got EOF
Unclosed paren	`"(end"`	expected RPAREN, got EOF
Empty branch	`"&{}"`	branch must have at least one choice
Empty selection	`"+{}"`	select must have at least one choice
Missing colon	`"&{m end}"`	expected COLON after label 'm'
Keyword as rec var	`"rec rec . end"`	'rec' is a keyword
end as rec var	`"rec end . end"`	'end' is a keyword
Dangling dot	`"m ."`	unexpected token EOF
Unexpected token	`"}"`	unexpected token RBRACE
Trailing after seq	`"m . }"`	unexpected token RBRACE after expression

Each `ParseError` carries a `pos` field that the annotation processor maps to the source location of the `@Session` annotation, so the developer sees a red underline in their IDE at the right place.

4 Stage 2: Termination Checking

Every recursive session type $\mu X.\, S$ must have an *exit path*—a branch or selection arm that does not recursively refer back to X. Without an exit path, the protocol describes an infinite loop with no way to reach **end**, which is not physically implementable.

4.1 Termination Error Examples

- **Trivial loop**: `rec X . X`
  The body is just the variable X itself—immediate infinite recursion with no choices at all.
- **All branches loop**: `rec X . &{loop: X}`
  There is a branch, but every arm leads back to X. The client has no choice that terminates.
- **All selections loop**: `rec X . ⊕{OK: X, ERR: X}`
  The object makes the choice, and every option loops. Even though the client doesn't control it, the protocol never ends.
- **Nested non-termination**: `rec X . &{a: rec Y . Y, b: X}`
  Variable X has an exit (branch b), but the inner $\mu Y.\, Y$ has none. The error reports variable Y.
- **Sequence then recurse**: $\mu X.\, X$. **end**
  The sequencing puts X on the left, which unfolds forever before reaching **end**.

A *valid* recursive type needs at least one path to **end**:

```
rec X . &{next: X, done: end}
```

Here, the `done` branch provides the exit.

5 Stage 3: Well-Formed Parallel (WF-Par)

The parallel constructor $S_1 \| S_2$ requires three well-formedness conditions from the specification (§5.1):

1. **No cross-branch variables.** A recursion variable bound in one branch must not appear free in the other. Violation:
   ```
   (X || rec X . X)
   ```
   Variable X is free on the left but bound on the right.
2. **No nested parallel.** The $\|$ constructor must not appear inside another $\|$. Violations:
   ```
   ( (a.end || b.end) || c.end )
   ( a.end || (b.end || c.end) )
   ```
   Both nest parallel inside parallel.
3. **Both branches terminate.** Each branch of the parallel must independently reach **end**. Violation:
   ```
   (rec X . X || end)
   ```
   The left branch is non-terminating.

6 Stage 4: State-Space Construction

Stages 1–3 operate on the AST. Stage 4 unfolds the AST into a *labeled transition system* (LTS): a directed graph where nodes are protocol states and edges are labeled transitions.

- **Branch** $\&\{m_1 : S_1, \ldots, m_n : S_n\}$ creates n outgoing transitions from the current state, each labeled m_i and leading to the state space of S_i. These are `METHOD` transitions—the client chooses which to call.
- **Selection** $\oplus\{l_1 : S_1, \ldots, l_n : S_n\}$ also creates n outgoing transitions, but they are `SELECTION` transitions—the object decides which path to take.
- **Parallel** $(S_1 \| S_2)$ constructs the product $L(S_1) \times L(S_2)$, ordered componentwise. A product state (s_1, s_2) has transitions from both branches independently.
- **Recursion** $\mu X.\, S$ allocates a placeholder for X, builds the body, then merges back-edges to create cycles.

The result is a `StateSpace` record with a `top` (initial state), `bottom` (**end**), states, and transitions. This stage rarely fails in isolation—syntactically valid, terminating, well-formed types always produce valid state spaces.

7 Stage 4.5: Object Conformance

This stage ensures that the Java class's method return types match the session type's selection labels. When a method m directly precedes a selection $\oplus\{l_1 : S_1, \ldots, l_n : S_n\}$, the method $m()$ must return an enum (or boolean) whose values cover $\{l_1, \ldots, l_n\}$.

DRAFT

7.1 Conformance Error Examples

- **Void method before selection:**

```
@Session("open . +{OK: end, ERR: end}")
class FileHandle {
    void open() { ... } // ERROR: must return enum
}
```

Error: `Method 'open' must return an enum or boolean to select from [ERR, OK].`

- **Enum missing a label:**

```
@Session("m . +{A: end, B: end}")
class Proto {
    enum Result { A } // missing B!
    Result m() { return Result.A; }
}
```

Error: `Method 'm' returns enum Result which is missing selection labels [B].`

- **Wrong return type:**

```
@Session("m . +{A: end, B: end}")
class Proto {
    int m() { return 0; } // int is not enum/boolean
}
```

Error: `Method 'm' must return an enum or boolean to select from [A, B].`

- **Boolean with non-boolean labels:**

```
@Session("m . +{A: end, B: end}")
class Proto {
    boolean m() { return true; } // covers {TRUE,FALSE}, not {A,B}
}
```

Error: `Method 'm' returns boolean (covers {TRUE, FALSE}) but session type expects selection labels [A, B].`

Implementation note. The conformance checker extracts the method→selection mapping from the *AST*, not the state space. This avoids false positives in protocols with parallel composition, where the product state space may contain states that combine methods from one branch with selections from another.

8 Stage 5: Lattice Checking

The central theorem of the research programme states that every well-formed session type state space forms a *lattice*: it has a top (initial state), a bottom (**end**), and every pair of states has a unique meet and join under the reachability ordering.

This stage verifies the lattice property. It first quotients the state space by strongly connected components (to handle cycles from recursion), then checks:

1. Top reachability: the initial state reaches all others.
2. Bottom reachability: all states reach the terminal state.
3. All pairwise meets exist.
4. All pairwise joins exist.

A violation would indicate either a bug in the theory or a state-space construction error. In practice, this check has never failed on a well-formed input—it serves as a confidence-building assertion.

If it did fail, the error would include a counterexample: two states and which binary operation (meet or join) is missing.

9 Stage 6: Thread Safety

For types with parallel composition $S_1 \parallel S_2$, methods from S_1 may execute concurrently with methods from S_2. Stage 6 checks that all such cross-branch pairs are *compatible* according to a four-level concurrency lattice:

$$\texttt{SHARED} > \texttt{SYNC}, \texttt{READ_ONLY} > \texttt{EXCLUSIVE}$$

Two methods are compatible iff neither is `EXCLUSIVE`. Methods are classified by annotations (`@Shared`, `@ReadOnly`) or default to `EXCLUSIVE`.

9.1 Thread Safety Error Example

```
@Session("open . (read.end || write.end)")
class File {
    void open() { ... }
    void read() { ... } // default: EXCLUSIVE
    void write() { ... } // default: EXCLUSIVE
}
```

Error: `read (EXCLUSIVE) and write (EXCLUSIVE) may execute concurrently but are not compatible.`

Fix: annotate both with `@ReadOnly` or `@Shared`.

Note: Selection labels (enum return values like `OK`, `REJECT`) are *not* treated as methods. They are excluded from the thread-safety analysis because they represent internal object decisions, not client-callable operations.

10 Stage 7: Typestate Checking

After all `@Session` classes are validated, the processor scans *every class in the compilation unit* for code that uses session-typed objects. It extracts method call sequences and simulates them against the state space.

10.1 Selection-Aware Checking

When the state machine reaches a pure *selection state*—one where only the object can act (`SELECTION` transitions, no `METHOD` transitions)—the checker *auto-advances* by trying each selection branch to find one where the client's next call is valid. This reflects the runtime semantics: the method that preceded the selection returns an enum, and the client inspects it.

For example, with session type `open . ⊕{OK: read.end, ERR: end}` and client code:

```
FileHandle f = new FileHandle();
f.open();    // now at selection state {OK, ERR}
f.read();    // auto-advance: OK branch enables read
```

The checker recognizes that `read()` is valid via the `OK` branch and advances accordingly.

10.2 Switch-Aware Extraction

The method call extractor builds a *call tree* (not just a flat list) by detecting `switch` statements on enum return values:

```
ATM.PinResult r = atm.enterPIN();
switch (r) {
    case AUTH -> { atm.checkBalance(); atm.ejectCard
        (); }
    case REJECTED -> { atm.ejectCard(); }
}
```

The switch produces a `SelectionSwitch` node in the call tree. Each branch is checked independently against the corresponding selection transition, then any code after the switch is verified from each branch's final state.

10.3 Typestate Error Examples

- **Method called out of order:**

```
FileHandle f = new FileHandle();
f.read(); // ERROR: open must come first
```

  Error: `Method 'read' is not enabled on 'f' in state 0 (enabled: [open]).`

- **Using object after end:**

```
FileHandle f = new FileHandle();
f.open(); f.read(); f.close();
f.open(); // ERROR: already at end state
```

  Error: `Method 'open' is not enabled on 'f' in state 4 (enabled: []).`

- **Not reaching end state:**

```
void handle(FileHandle f) {
    f.open();
    // method returns without calling read+
        close
}
```

  Error: `Variable 'f' is not in terminal state at end of method.`

11 The Complete Error Hierarchy

Table 2 summarizes the pipeline with input, check, and error type for each stage.

Stages 1–6 run on the `@Session`-annotated class itself. A failure at any stage halts the pipeline for that class (but other classes continue independently). Stage 7 runs only after all classes pass Stages 1–6, and it scans *all* classes in the compilation unit—including test classes and client code.

Table 2. Pipeline stages, their inputs, and error types.

Stage	Input	Check	Error type
1	String	Syntax	`ParseError`
2	AST	Exit paths	`TerminationResult`
3	AST	WF-Par rules	`WFParallelResult`
4	AST	Construction	`Exception`
4.5	AST+Class	Return types	`ConformanceError`
5	StateSpace	$\wedge/\vee$ exist	`LatticeResult`
6	AST+Class	Concurrent pairs	`ThreadSafetyResult`
7	Client code	Protocol trace	`TypestateError`

12 Ordering Rationale

The stages are ordered by *dependency* and *cost*:

1. **Parse before anything else.** Without a valid AST, nothing can proceed.
2. **Termination before state space.** A non-terminating type would produce an infinite (or malformed) state space.
3. **WF-Par before state space.** The product construction assumes no cross-branch variables and no nested parallel.
4. **State space before lattice/conformance/safety.** All subsequent stages need the state space.
5. **Conformance before lattice.** Conformance is a fast structural check; lattice checking is more expensive. Catching return-type mismatches early saves time.
6. **Lattice before thread safety.** If the state space isn't a lattice, the theory doesn't apply, and thread-safety analysis is meaningless.
7. **Thread safety before typestate.** If parallel methods are unsafe, checking client code against the protocol provides a false sense of correctness.
8. **Typestate last.** It requires all `@Session` classes to be validated first, since client code may reference multiple session-typed objects.

13 A Successful Run

When all seven stages pass, the processor emits a compiler `NOTE`:

```
Session type for com.example.ATM verified:
  7 states, 11 transitions, lattice OK,
  6 methods classified
```

This confirms that the session type was parsed, is terminating, well-formed, forms a lattice, has conformant return types, and all concurrent method pairs are compatible. Any client code that follows the protocol (as verified in Stage 7) is guaranteed to use the object correctly.

14 Conclusion

The Bica Reborn pipeline demonstrates that session type checking can be integrated into a standard Java build with

DRAFT

no special compiler or runtime. The seven stages provide progressively deeper guarantees: from syntactic well-formedness, through algebraic properties (lattice), to behavioral correctness (typestate). Each stage produces actionable error messages, and the ordering ensures that failures are reported at the earliest possible point.

The pipeline currently validates 34 benchmark protocols and passes 1,052 unit tests in the Bica Reborn library, plus 119 integration tests in the tutorial benchmark suite. The complete source code is available at https://bica.zuacaldeira.com.

DRAFT